

MACHINE LEARNING FOR DYNAMIC FACTOR ROTATION ACROSS MARKET REGIMES

Authors: Guoyan (Aiden) Li: 20940209, Honghao (Samuel) Shen: 20927036 and
Vito Zhao: 20931327

Table of Contents

Abstract	2
1. Introduction	3
2. Methodology	4
2.1 Models	4
2.2 Rolling Window Framework	4
2.3 Regime Classification	5
2.4 Evaluation Metrics	5
3. Data and Experiment Design	7
3.1 Fama-French Six Factors	7
3.2 Goyal-Welch Macroeconomic Predictors	7
3.3 VIX Data	8
3.4 Data Preprocessing	8
4. Result and Analysis	9
4.1 Out-of-Sample R^2	9
4.2 Sharpe Ratio	10
4.3 Hit Ratio	12
4.4 Why Negative R^2 but Positive Sharpe?	14
5. Conclusion	16
References	18
Appendix	19

Abstract

This paper implements three machine learning models: LASSO regression, Random Forest, and Neural Network to predict Fama-French six-factor returns using Goyal-Welch (2024) macroeconomic predictors. We use a 60-month rolling window out-of-sample framework from 1995 to 2023 and classify market regimes using the CBOE Volatility Index (VIX index). Our results show that all three models produce negative out-of-sample R^2 , meaning none of them beats the simple historical mean in terms of prediction accuracy. However, all models achieve hit ratios above 0.50 on most factors, suggesting they can still predict return direction reasonably well. More importantly, a long-short factor rotation strategy built from model predictions achieves annualized Sharpe Ratios of around 0.70–0.78 during low-volatility periods, compared to near zero during high-volatility periods. This confirms what we hypothesized in Stage 1 that machine learning models for factor rotation work much better in calm markets than in turbulent ones. Projects 1 and 2 are highly related, therefore the reader is referred to our GPR # 1 report for related work.

1. Introduction

Factor investing is a well-established approach in quantitative finance. The basic idea is that certain characteristics like value, size, profitability, or momentum can help predict which assets will outperform. Traditional factor models use linear regressions, which are simple and interpretable, but they might miss more complex patterns in the data. In recent years, machine learning methods have been proposed to capture these nonlinear relationships.

Our Stage 1 report reviewed three papers on this topic. Kozak, Nagel, and Santosh (2020) showed that LASSO can stabilize cross-sectional return prediction by removing irrelevant predictors. Wolff and Neugebauer (2019) found that tree-based models like Random Forest do not consistently beat linear models at the aggregate level but suggested that these advantages might be "state-dependent." Chen, Pelger, and Zhu (2019) demonstrated that neural networks can capture complex nonlinear patterns in asset pricing, though they require large samples and careful regularization. One thing missing from all three papers is an explicit look at how model performance changes across different market conditions. Financial markets behave very differently during calm periods versus crisis periods, and it seems reasonable to think that factor predictability might also differ, and this is the main gap we try to address in Stage 2.

Our three research questions are:

1. Do nonlinear machine learning models (Random Forest, Neural Network) outperform LASSO in predicting factor returns?
2. Does model performance differ systematically between low-volatility and high-volatility market regimes?
3. Can machine learning-based factor rotation strategies generate real economic value, and does this depend on the market regime?

2. Methodology

2.1 Models

We implement three models that represent different points on the complexity spectrum.

LASSO Regression: LASSO adds an L1 penalty to OLS regression, which forces some coefficients to shrink to exactly zero. This is useful for variable selection when we have many predictors. We select the regularization parameter λ through 5-fold cross-validation. Following Kozak et al. (2020), LASSO serves as our linear benchmark.

Random Forest: We use an ensemble of 200 regression trees ($\text{ntree} = 200$), with 4 predictors randomly considered at each split ($\text{mtry} = 4$, roughly the square root of 11 predictors). Random Forest captures nonlinear interactions between predictors and is less sensitive to outliers than individual trees. This is the tree-based approach motivated by Wolff and Neugebauer (2019).

Neural Network: We implement a single hidden-layer feedforward neural network with 5 hidden nodes, a linear output layer (since this is a regression task), and L2 weight decay regularization ($\text{decay} = 0.01$) to reduce overfitting. We allow up to 200 training iterations. This is our approximation of the deep learning approach discussed in Chen et al. (2019), but simplified for feasibility.

2.2 Rolling Window Framework

To avoid look-ahead bias, we use a 60-month (5-year) rolling window. At each month t , we train the model on data from month $t-60$ to $t-1$, then predict the return for month t . We never let future

data into the training set. This gives us out-of-sample predictions from 1995 onward, covering approximately 347 months. A random seed of 423 is set for reproducibility.

2.3 Regime Classification

We classify each month as either a low-volatility or high-volatility regime using the VIX index. Months where VIX is at or below the full-sample median (~17–18) are labeled low-volatility; months above the median are high-volatility. This gives us an approximately balanced split and avoids the complexity of formal regime-switching models.

2.4 Evaluation Metrics

We evaluate models using three metrics, each capturing a different aspect of performance.

Out-of-Sample R^2 (OOS R^2) measures whether the model beats the historical mean in terms of mean squared error:

$$R_{OOS}^2 = 1 - \frac{\text{MSE}(\text{model})}{\text{MSE}(\text{historical mean})}$$

Positive values mean the model outperforms the benchmark; negative values mean it does worse.

Hit Ratio measures the proportion of months where the model correctly predicts the sign (direction) of the factor return. A value above 0.5 suggests the model is doing better than random guessing.

Sharpe Ratio of Long-Short Strategy captures the economic value. Each month, we go long the factor with the highest predicted return and short the factor with the lowest predicted return. The annualized Sharpe Ratio is then:

$$\text{SR} = \frac{\bar{r}_{LS}}{\sigma_{LS}} \times \sqrt{12}$$

This tells us how much return we earn per unit of risk taken by following the model's factor rotation signal.

3. Data and Experiment Design

3.1 Fama-French Six Factors

Our response variables are six monthly Fama-French factor returns: Market excess return (MKT), Size (SMB), Value (HML), Profitability (RMW), Investment (CMA), and Momentum (MOM). We download these from Kenneth French's Data Library using the frenchdata R package and convert them from percentages to decimals. The full sample covers January 1990 to December 2023. After applying a 60-month rolling window, the out-of-sample prediction period runs from February 1995 to December 2023, yielding approximately 347 monthly observations

3.2 Goyal-Welch Macroeconomic Predictors

Our predictors come from the Goyal–Welch macroeconomic predictor dataset, originally introduced by Goyal and Welch (2008) and later updated on Amit Goyal's personal website. We use the 2024 update of the dataset, while restricting our sample to end in December 2023. This dataset was also adopted by Wolff and Neugebauer (2019) in their tree-based equity market prediction study, which provides a direct methodological link to the literature we reviewed in Stage 1 and allows for a more comparable evaluation of model performance across studies. The specific Excel file used in this study is available on Amit Goyal's website and contains all data from Goyal and Welch (2008) and Goyal and Welch (2024) in a single file. We select 11 variables that are most used as return predictors in the empirical finance literature: dividend-price ratio (dp), earnings-price ratio (ep), book-to-market ratio (bm), T-bill rate (tbl), long-term bond yield (lty), term spread (tms), default yield spread (dfy), default return spread (dfr), inflation (infl), long-term bond return (ltr), and stock return variance (svar). These variables cover valuation, interest rate, and

macroeconomic cycle signals.

3.3 VIX Data

Monthly VIX values are downloaded from Yahoo Finance using the quantmod package. We convert the daily series to monthly by taking the first-of-month closing price. It is worth noting that VIX data is only available from 1993; however, since all observations prior to 1995 are removed before evaluation, this gap does not affect the results in practice.

3.4 Data Preprocessing

We apply the following preprocessing steps to the predictor variables:

- **One-month lag:** All predictors are lagged by one month before merging with factor returns. This ensures that X_{t-1} is used to predict Y_t preventing any look-ahead bias. The first observation is dropped after lagging.
- **Winsorization at 1% & 99%:** Extreme outliers in the predictors are clipped to the 1st and 99th percentile values to reduce their influence on model training.
- **Standardization:** All predictors are scaled to zero mean and unit variance. This is required for LASSO (so that all coefficients are on the same scale) and helps neural network convergence.
- **Regime Classification:** Each observation is assigned a volatility regime label based on the VIX-based classification described in Section 2.3. This regime label is not used as a predictor during model training but applied only during evaluation to split the out-of-sample predictions into low-volatility and high-volatility subsamples.

4. Result and Analysis

4.1 Out-of-Sample R^2

Table 1 shows the OOS R^2 for each model and factor over the full sample. All values are negative across the board, which means none of the models can beat the historical mean benchmark in terms of prediction accuracy. At first glance, this might seem like a failure. However, this is consistent with what the literature finds, as monthly factor return prediction is notoriously difficult, and even well-specified models typically fail to outperform the historical mean in out-of-sample tests (Welch & Goyal, 2008).

Model	MKT	SMB	HML	RMW	CMA	MOM
LASSO	-0.177	-0.254	-0.210	-0.082	-0.181	-0.044
RF	-0.175	-0.112	-0.129	-0.060	-0.091	-0.113
NN	-0.536	-0.453	-0.297	-0.324	-0.250	-0.547

Table 1: OOS R^2 by Model and Factor

Comparing across models, Random Forest tends to have the least negative R^2 , especially for RMW and CMA. LASSO is in the middle. Neural Network performs worst by a significant margin, likely because a shallow one-hidden-layer network with only 60 months of training data tends to overfit. This is consistent with the warning from Chen et al. (2019) that neural networks require large datasets to work well.

To see whether predictive accuracy differs across market conditions, Table 2 breaks down the average OOS R^2 by volatility regime.

Model	Low-Volatility	High-Volatility
LASSO	-0.145	-0.188
RF	-0.237	-0.108
NN	-0.477	-0.423

Table 2: Average OOS R^2 by Regime

One interesting pattern here is that RF performs better (less negative) in high-volatility regimes. This could be because factor return dispersion is higher during volatile periods, making it somewhat easier for the model to rank factors, even if it cannot predict exact values.

4.2 Sharpe Ratio

While OOS R^2 measures magnitude accuracy, it does not tell us whether the models generate real economic value. To assess this, Table 3 reports the annualized Sharpe Ratio of the long-short factor rotation strategy for each model across the full sample and by regime.

Model	Full Sample	Low-Volatility	High-Volatility
LASSO	0.219	0.705	0.047
RF	0.198	0.777	-0.031
NN	0.221	0.699	0.045

Table 3: Annualized Sharpe Ratio of Long-Short Factor Rotation Strategy

This is the most striking result in our study. Despite having negative R^2 across the board, all three models generate Sharpe Ratios around 0.70–0.78 in low-volatility regimes. In high-volatility regimes, the Sharpe Ratio drops to nearly zero or negative. To put the low-volatility Sharpe Ratio in context, a value of 0.7–0.8 is comparable to many actively managed equity strategies and is considered economically meaningful in the academic literature. The fact that we achieve this purely from simple macroeconomic predictors and standard machine learning models is encouraging.

The regime gap is large: moving from low-vol to high-vol, the Sharpe Ratio drops by roughly 0.7 units across all three models. This strongly suggests that the macroeconomic predictors carry useful relative factor ranking information in calm markets but lose their signal during turbulent periods when factor relationships become unstable.

Figure 1 plots the cumulative returns of all three strategies over time with regime shading. The

green (low-vol) bands are where most of the return accumulation happens, while red (high-vol) bands correspond to flat or declining periods. This visual pattern makes the regime asymmetry very clear.



Figure 1: Cumulative returns of the long-short factor rotation strategy for LASSO, RF, and NN over the out-of-sample period (1995–2023). Green shading indicates low-volatility regimes and red shading indicates high-volatility regimes based on VIX classification; most return accumulation visibly concentrates in low-volatility periods.

To examine how this plays out over time, Figure 2 shows the rolling 12-month Sharpe Ratio for each model. The figure confirms that good performance is not constant. It appears in clusters that overlap heavily with low-volatility periods, while the strategy tends to break down during high-volatility episodes such as the 2008 financial crisis and the 2020 COVID shock.

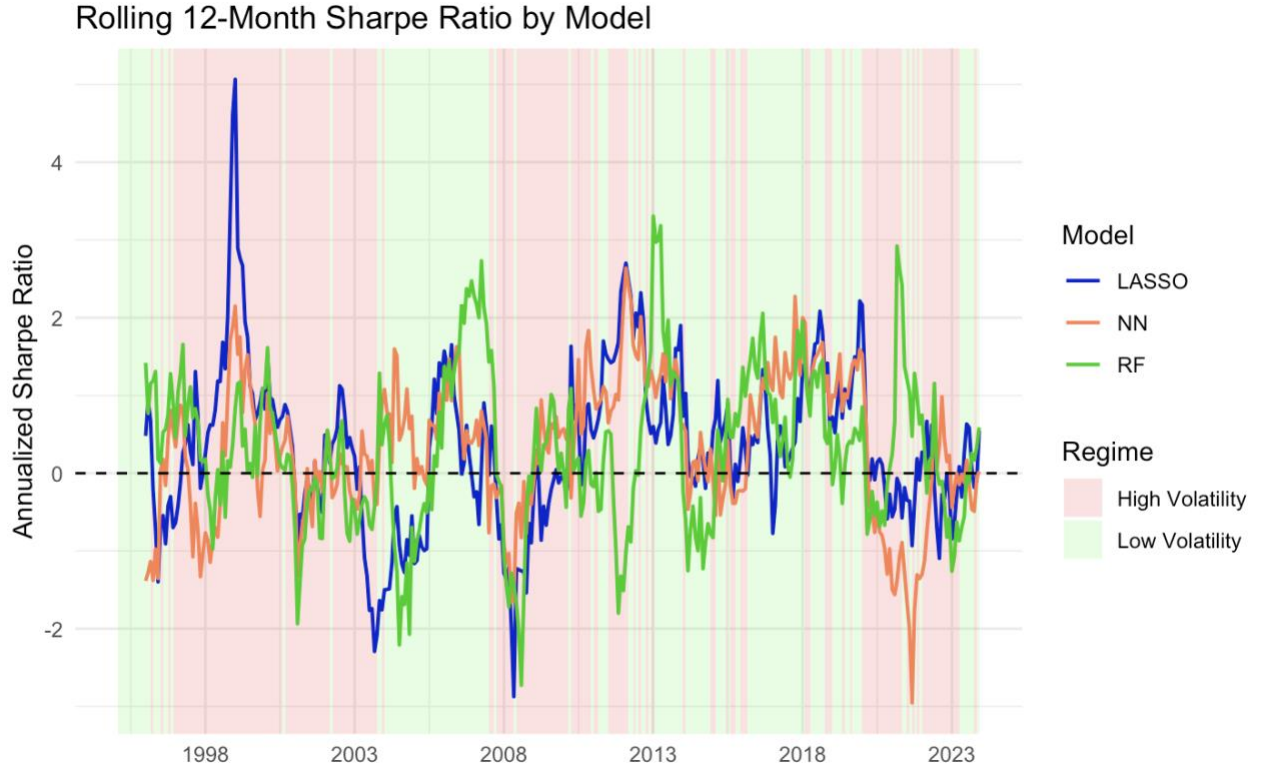


Figure 2: Rolling 12-month annualized Sharpe Ratio of the long-short strategy for each model (1995–2023), with the dashed line marking zero. Strategy performance is highly time-varying and tends to be positive during low-volatility periods while deteriorating during high-volatility episodes.

4.3 Hit Ratio

To understand why the strategy generates positive Sharpe despite negative R^2 , we look at the hit ratio. Table 4 reports the hit ratio for each model and factor over the full sample.

Model	MKT	SMB	HML	RMW	CMA	MOM
LASSO	0.594	0.513	0.516	0.516	0.487	0.579
RF	0.559	0.478	0.504	0.499	0.542	0.499
NN	0.559	0.487	0.545	0.510	0.470	0.553

Table 4: Hit Ratio by Model and Factor (Full Sample)

Hit ratios are mostly above 0.5, which means the models are getting the direction right often. This helps explain why the long-short strategy still works despite poor magnitude prediction. A long-short strategy only needs to correctly rank which factor does better, not predict the exact return level. LASSO achieves the highest hit ratios overall, particularly for MKT (0.594) and MOM

(0.579), suggesting that linear macro-factor relationships are sufficient for directional prediction.

Table 5 further breaks down the average hit ratio by regime.

Model	Low-Volatility	High-Volatility
LASSO	0.559	0.512
RF	0.522	0.506
NN	0.542	0.503

Table 5: Average Hit Ratio by Model and Regime

Hit ratios are consistently higher in low-volatility regimes across all models, again confirming that regime matters. The pattern is consistent with the Sharpe Ratio results in Table 3, as models predict direction more reliably when markets are calm.

Figure 3 shows how the rolling 12-month hit ratio evolves over time. Although all three models fluctuate around the 0.5 random guessing benchmark, hit ratios tend to be higher during low-volatility periods and occasionally fall below 0.5 during more turbulent episodes, such as 2008–2009 and 2020.

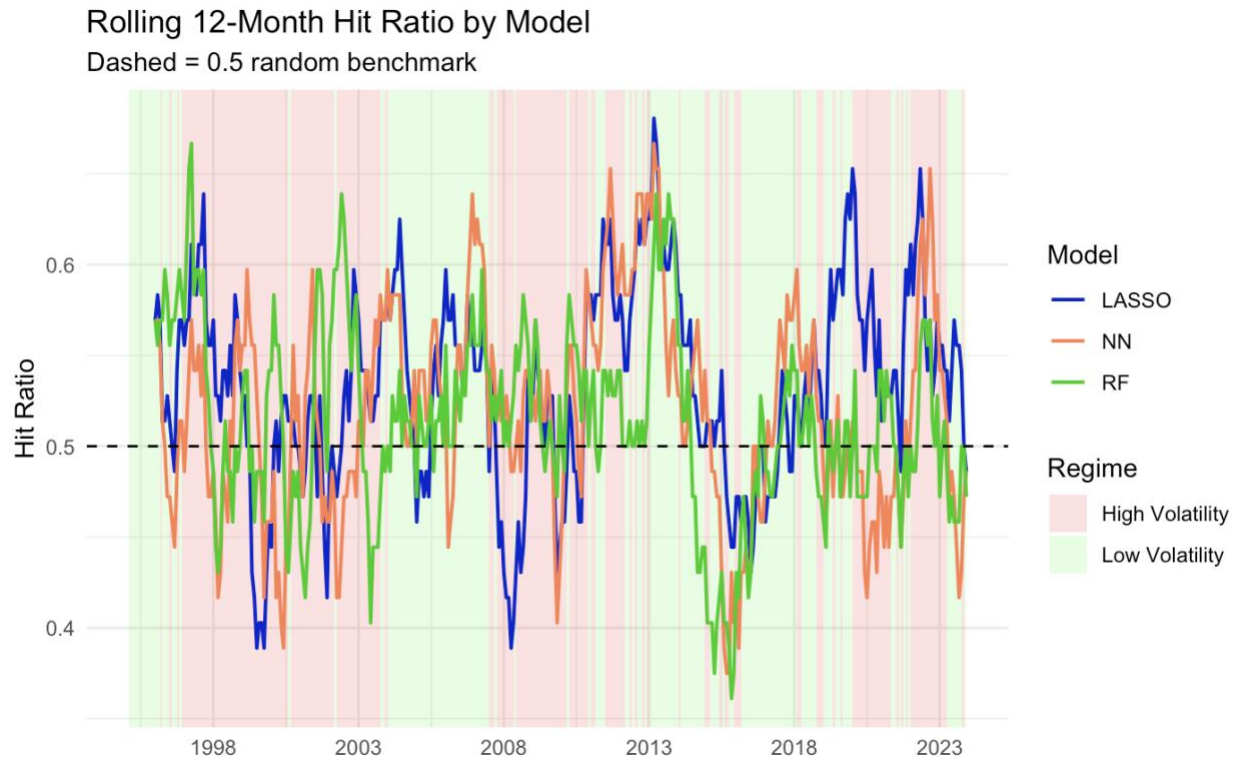


Figure 3: Rolling 12-month hit ratio for the LASSO, RF, and NN models from 1995 to 2023. The dashed horizontal line marks the 0.5 benchmark, while the background shading indicates low and high volatility regimes classified by VIX.

4.4 Why Negative R^2 but Positive Sharpe?

The results in Tables 1 to 5 raise an apparent contradiction that is worth addressing directly. R^2 penalizes the model for every prediction that deviates from the actual value, whether in direction or magnitude. But a long-short factor rotation strategy only cares about the relative ranking of factors, specifically which factor to go long and which to go short. A model can be systematically wrong about how large the returns will be (leading to negative R^2) while still being right about the ordering of factors (leading to positive Sharpe Ratio).

In other words, the Goyal-Welch predictors seem to carry more information about which factors will do better relative to each other than about their exact return levels. This is an important and practically useful finding. We do not need perfect return forecasts to build a profitable rotation strategy, just need reasonably reliable relative signals instead.

Figure 4 provides one more piece of evidence for this interpretation. It shows the rolling 12-month OOS R^2 over time, which is highly variable with large negative spikes in the early sample period (1995 to 2003) and values closer to zero in later years. Even in periods when R^2 is deeply negative, the strategy in Figure 1 still accumulates positive returns, further confirming that R^2 is not the right metric for evaluating the economic value of factor rotation models.

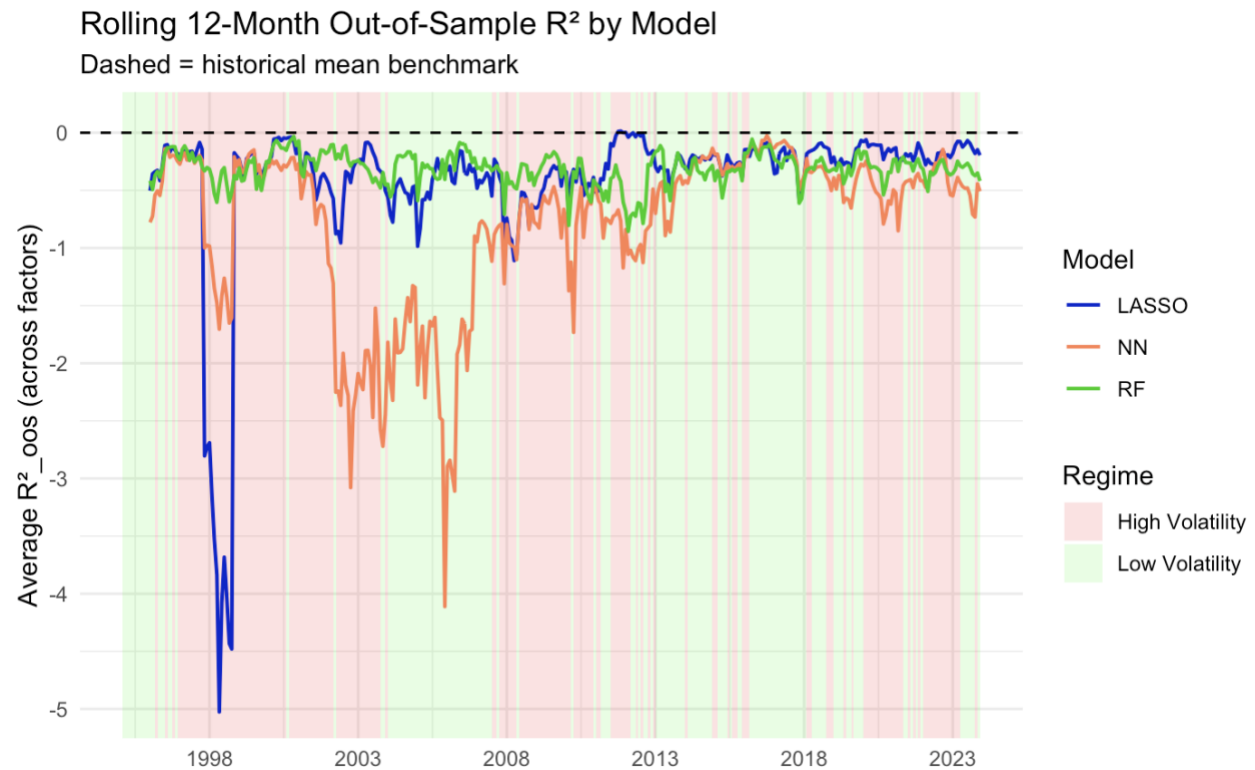


Figure 4: Rolling 12-month average out-of-sample R^2 across all six factors for each model from 1995 to 2023, with the dashed line marking the historical mean benchmark at zero. All models produce negative R^2 throughout most of the sample, and the extreme negative values in the early sample may partly reflect instability associated with relatively short rolling estimation histories.

5. Conclusion

In this paper, we implemented LASSO, Random Forest, and Neural Network for Fama-French six-factor return prediction using Goyal-Welch macroeconomic predictors and evaluated their performance across VIX-defined volatility regimes from 1995 to 2023. And here are our findings: First, all three models fail to beat the historical mean in terms of OOS R^2 . This is consistent with the broader literature and confirms that factor return magnitude prediction is a very difficult task, even with machine learning. Random Forest performs best among the three; Neural Network performs worst, likely due to overfitting in small training samples.

Second, despite negative R^2 , all models can predict factor return direction (hit ratio > 0.5 in most cases), and this directional accuracy is enough to build profitable long-short strategies in low-volatility regimes. The Sharpe Ratio of around 0.70–0.78 during low-volatility periods is economically meaningful, while in high-volatility periods it drops to near zero. This is our most important finding, and it directly confirms the regime-dependence hypothesis we proposed in Stage 1.

Third, the difference between R^2 and Sharpe Ratio results highlights that statistical accuracy metrics do not fully capture the practical value of factor timing models. For long-short rotation strategies, directional accuracy matters more than magnitude accuracy.

Our results support the view from Wolff and Neugebauer (2019) that machine learning advantages are state-dependent and extend their work by explicitly quantifying how much of that advantage is concentrated in low-volatility periods. From a practical standpoint, our findings suggest that factor rotation strategies should be most active during calm market environments and should reduce exposure during high-volatility regimes when the macro signals become unreliable.

One key limitation of our study is the relatively simple neural network architecture we use. A

deeper network with more training data might capture richer nonlinear patterns. Additionally, we do not account for transaction costs, which could reduce realized Sharpe Ratios. Future work could explore whether combining regime signals directly into the model training process (rather than just using them for evaluation) improves out-of-sample performance.

References

- Chen, L., Pelger, M., & Zhu, J. (2019). *Deep learning in asset pricing* (SSRN Scholarly Paper No. 3350138). Social Science Research Network.
https://papers.ssrn.com/sol3/papers.cfm?abstract_id=3350138
- Goyal, A., & Welch, I. (2008). *A comprehensive look at the empirical performance of equity premium prediction. Review of Financial Studies*, 21(4), 1455–1508.
<https://academic.oup.com/rfs/article-abstract/21/4/1455/1565737?redirectedFrom=fulltext>
- Goyal, A., Welch, I., & Zafirov, A. (2024). *A comprehensive 2022 look at the empirical performance of equity premium prediction. Review of Financial Studies*, 37(11), 3490–3557.
<https://academic.oup.com/rfs/article/37/11/3490/7749383>
- Goyal, A. (2024). *All data up to 2024 (includes data from GW (2008) and GWZ (2024) in one file)* [Excel data file]. Amit Goyal's personal website.
<https://sites.google.com/view/agoyal145>
- Kozak, S., Nagel, S., & Santosh, S. (2017). *Shrinking the cross section* (NBER Working Paper No. 24070). National Bureau of Economic Research.
https://papers.ssrn.com/sol3/papers.cfm?abstract_id=2945663
- Wolff, D., & Neugebauer, U. (2019). *Tree-based machine learning approaches for equity market predictions. Journal of Asset Management*, 20(4), 273–288.
<https://link.springer.com/article/10.1057/s41260-019-00125-5>

Appendix: AFM423 GPR2 R code

1: Data Collection

```
# 1.0 Install & Load Packages  
# install.packages(c("frenchdata", "quantmod", "tidyverse", "readxl", "lubridate"))  
  
library(frenchdata)  
library(quantmod)
```

```
## Loading required package: xts
```

```
## Loading required package: zoo
```

```
##
```

```
## Attaching package: 'zoo'
```

```
## The following objects are masked from 'package:base':
```

```
##
```

```
##      as.Date, as.Date.numeric
```

```
## Loading required package: TTR
```

```
## Registered S3 method overwritten by 'quantmod':
```

```
##      method      from
```

```
##      as.zoo.data.frame zoo
```

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
```

```
## v dplyr      1.1.4      v readr      2.1.6
```

```
## v forcats    1.0.1      v stringr    1.6.0
```

```
## v ggplot2    4.0.1      v tibble     3.3.1
```

```
## v lubridate  1.9.5      v tidyr      1.3.2
```

```
## v purrr      1.2.1
```

```
## -- Conflicts ----- tidyverse_conflicts() --
```

```
## x dplyr::filter() masks stats::filter()
```

```
## x dplyr::first()  masks xts::first()
```

```
## x dplyr::lag()    masks stats::lag()
```

```
## x dplyr::last()   masks xts::last()
```

```
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```

library(readxl)
library(lubridate)

# 1.1 Fama-French 6 Factor Data
# Source: Kenneth French Data Library (via frenchdata package)

ff6 <- download_french_data("Fama/French 5 Factors (2x3)")

## New names:
## New names:
## * ' ' -> '...1'

ff_mom <- download_french_data("Momentum Factor (Mom)")

## New names:
## New names:
## * ' ' -> '...1'

# Extract monthly data
ff6_monthly <- ff6$subsets$data[[1]] %>%
  mutate(date = as.Date(paste0(date, "01"), format = "%Y%m%d")) %>%
  rename(MKT = `Mkt-RF`) %>%
  select(date, MKT, SMB, HML, RMW, CMA, RF)

mom_monthly <- ff_mom$subsets$data[[1]] %>%
  mutate(date = as.Date(paste0(date, "01"), format = "%Y%m%d")) %>%
  rename(MOM = Mom) %>%
  select(date, MOM)

# Merge FF factors
ff_data <- ff6_monthly %>%
  left_join(mom_monthly, by = "date") %>%
  filter(date >= as.Date("1990-01-01"),
         date <= as.Date("2023-12-01")) %>%
  mutate(across(c(MKT, SMB, HML, RMW, CMA, MOM, RF), ~ . / 100))

cat("FF Data:", nrow(ff_data), "monthly observations\n")

## FF Data: 408 monthly observations

# 1.2 Goyal-Welch Macroeconomic Predictors
# Download Data2024.xlsx from Amit Goyal's website https://sites.google.com/view/agoyal145
# https://docs.google.com/spreadsheets/d/10_nk0kJPuq4eZgNl-1ys63PzhbnM3S2y/edit?usp=
# sharing&ouid=113571510202500088860&rtipof=true&sd=true

gw_raw <- read_excel("/Users/vitozhao/Downloads/Data2024.xlsx", sheet = "Monthly")

gw_data <- gw_raw %>%
  mutate(date = as.Date(paste0(yyyymm, "01"), format = "%Y%m%d")) %>%
  select(date,
         dp = `d/p`, # Dividend-price ratio

```

```

    ep   = `e/p`, # Earnings-price ratio
    bm   = `b/m`, # Book-to-market ratio
    tbl, # T-bill rate
    lty, # Long-term yield
    tms, # Term spread
    dfy, # Default yield spread
    dfr, # Default return spread
    infl, # Inflation
    ltr, # Long-term return
    svar # Stock variance
  ) %>%
  filter(date >= as.Date("1990-01-01"),
         date <= as.Date("2023-12-01"))

cat("GW Data:", nrow(gw_data), "monthly observations\n")

```

```
## GW Data: 408 monthly observations
```

```

# 1.3 VIX Data for Regime Classification
# Source: Yahoo Finance via quantmod

getSymbols("^VIX", src = "yahoo",
           from = "1990-01-01",
           to   = "2023-12-31",
           auto.assign = TRUE)

```

```
## [1] "VIX"
```

```

vix_xts <- to.monthly(VIX, indexAt = "firstof")
vix_monthly <- data.frame(
  date = as.Date(index(vix_xts)),
  VIX  = as.numeric(Cl(vix_xts))
) %>%
  filter(!is.na(VIX),
         date >= as.Date("1990-01-01"),
         date <= as.Date("2023-12-01"))

cat("VIX Data:", nrow(vix_monthly), "monthly observations\n")

```

```
## VIX Data: 408 monthly observations
```

```

# 1.4 Merge All Data

master_data <- ff_data %>%
  left_join(gw_data, by = "date") %>%
  left_join(vix_monthly, by = "date") %>%
  arrange(date)

cat("\nMissing values per column:\n")

```

```

##
## Missing values per column:

```

```
print(colSums(is.na(master_data)))
```

```
## date  MKT  SMB  HML  RMW  CMA  RF  MOM  dp  ep  bm  tbl  lty  tms  dfy  dfr
##      0    0    0    0    0    0    0    0    0    0    0    0    0    0    0    0
## infl  ltr svar  VIX
##      0    0    0    0
```

```
cat("\nFinal dataset:", nrow(master_data), "rows,",
    ncol(master_data), "columns\n")
```

```
##
## Final dataset: 408 rows, 20 columns
```

```
cat("Period:", as.character(min(master_data$date)),
    "to", as.character(max(master_data$date)), "\n")
```

```
## Period: 1990-01-01 to 2023-12-01
```

2: Data Preprocessing

```
# 2.1 Use Lag Predictors by 1 Month to avoid look-ahead bias
# X at time t predicts Y at time t+1
```

```
predictors <- c("dp", "ep", "bm", "tbl", "lty", "tms",
               "dfy", "dfr", "infl", "ltr", "svar")
```

```
factors <- c("MKT", "SMB", "HML", "RMW", "CMA", "MOM")
```

```
model_data <- master_data %>%
  mutate(across(all_of(predictors), ~ lag(., 1))) %>%
  filter(!is.na(dp)) # remove first row lost due to lagging
```

```
cat("After lagging predictors:", nrow(model_data), "rows\n")
```

```
## After lagging predictors: 407 rows
```

```
# 2.2 Winsorize Predictors at 1% / 99% to handle outliers
```

```
winsorize <- function(x, lower = 0.01, upper = 0.99) {
  q <- quantile(x, probs = c(lower, upper), na.rm = TRUE)
  pmax(pmin(x, q[2]), q[1])
}
```

```
model_data <- model_data %>%
  mutate(across(all_of(predictors), winsorize))
```

```
# 2.3 Standardize Predictors
```

```

model_data <- model_data %>%
  mutate(across(all_of(predictors), ~ scale(.)[, 1]))

# 2.4 Regime Classification using VIX
# Low volatility: VIX <= median
# High volatility: VIX > median

vix_median <- median(model_data$VIX, na.rm = TRUE)
cat("VIX median threshold:", round(vix_median, 2), "\n")

```

```
## VIX median threshold: 18
```

```

model_data <- model_data %>%
  mutate(regime = ifelse(VIX <= vix_median, "low_vol", "high_vol"),
         regime = factor(regime, levels = c("low_vol", "high_vol")))

cat("\nRegime distribution:\n")

```

```
##
## Regime distribution:
```

```
print(table(model_data$regime))
```

```
##
## low_vol high_vol
##      204      203
```

```
# 2.5 Summary Check
```

```

cat("\nFinal model_data:", nrow(model_data), "rows,",
    ncol(model_data), "columns\n")

```

```
##
## Final model_data: 407 rows, 21 columns
```

```
cat("Missing values:\n")
```

```
## Missing values:
```

```
print(colSums(is.na(model_data)))
```

```
##  date  MKT  SMB  HML  RMW  CMA  RF  MOM  dp  ep  bm
##    0    0    0    0    0    0    0    0    0    0    0
##  tbl  lty  tms  dfy  dfr  infl  ltr  svar  VIX regime
##    0    0    0    0    0    0    0    0    0    0
```

3: Model Definitions


```
# install.packages(c("glmnet", "randomForest", "nnet"))
library(glmnet)
```

```
## Loading required package: Matrix
```

```
##
```

```
## Attaching package: 'Matrix'
```

```
## The following objects are masked from 'package:tidyr':
```

```
##
```

```
##     expand, pack, unpack
```

```
## Loaded glmnet 4.1-10
```

```
library(randomForest)
```

```
## randomForest 4.7-1.2
```

```
## Type rfNews() to see new features/changes/bug fixes.
```

```
##
```

```
## Attaching package: 'randomForest'
```

```
## The following object is masked from 'package:dplyr':
```

```
##
```

```
##     combine
```

```
## The following object is masked from 'package:ggplot2':
```

```
##
```

```
##     margin
```

```
library(nnet)
```

```
# LASSO
```

```
fit_lasso <- function(X_train, y_train, X_test) {  
  cv_fit <- cv.glmnet(X_train, y_train, alpha = 1, nfolds = 5)  
  predict(cv_fit, newx = X_test, s = "lambda.min")[1]  
}
```

```
# Random Forest
```

```
fit_rf <- function(X_train, y_train, X_test) {  
  model <- randomForest(x = X_train, y = y_train,  
                        ntree = 200, mtry = 4)  
  predict(model, newdata = X_test)  
}
```

```
# Neural Network (MLP)
```

```
fit_nn <- function(X_train, y_train, X_test) {  
  model <- nnet(x = X_train, y = y_train,
```

```

        size = 5, linout = TRUE,
        decay = 0.01, maxit = 200,
        trace = FALSE)
predict(model, newdata = X_test)[1]
}

```

4: Rolling Window Prediction

```

# 4.0 Setup
set.seed(423)
train_window <- 60      # 60 months training window
n <- nrow(model_data)
dates <- model_data$date

# Empty matrices to store predictions (rows = time, cols = factors)
pred_lasso <- matrix(NA, nrow = n, ncol = length(factors),
                     dimnames = list(NULL, factors))
pred_rf <- matrix(NA, nrow = n, ncol = length(factors),
                  dimnames = list(NULL, factors))
pred_nn <- matrix(NA, nrow = n, ncol = length(factors),
                  dimnames = list(NULL, factors))

```

```

# 4.1 Rolling Window Loop
cat("Running rolling window predictions...\n")

```

```
## Running rolling window predictions...
```

```

for (t in (train_window + 1):n) {

  train_idx <- (t - train_window):(t - 1)

  X_train <- as.matrix(model_data[train_idx, predictors])
  X_test <- as.matrix(model_data[t, predictors])

  for (fac in factors) {
    y_train <- model_data[[fac]][train_idx]

    pred_lasso[t, fac] <- fit_lasso(X_train, y_train, X_test)
    pred_rf[t, fac] <- fit_rf(X_train, y_train, X_test)
    pred_nn[t, fac] <- fit_nn(X_train, y_train, X_test)
  }

  if (t %% 12 == 0) cat("Done:", as.character(dates[t]), "\n")
}

```

```

## Done: 1996-01-01
## Done: 1997-01-01
## Done: 1998-01-01
## Done: 1999-01-01
## Done: 2000-01-01

```

```
## Done: 2001-01-01
## Done: 2002-01-01
## Done: 2003-01-01
## Done: 2004-01-01
## Done: 2005-01-01
## Done: 2006-01-01
## Done: 2007-01-01
## Done: 2008-01-01
## Done: 2009-01-01
## Done: 2010-01-01
## Done: 2011-01-01
```

```
## Done: 2012-01-01
```

```
## Done: 2013-01-01
## Done: 2014-01-01
## Done: 2015-01-01
```

```
## Done: 2016-01-01
```

```
## Done: 2017-01-01
## Done: 2018-01-01
## Done: 2019-01-01
## Done: 2020-01-01
```

```
## Done: 2021-01-01
```

```
## Done: 2022-01-01
```

```
## Done: 2023-01-01
```

```
# 4.2 Collect results into one data frame
results <- model_data %>%
  select(date, regime, all_of(factors)) %>%
  rename_with(~ paste0("actual_", .), all_of(factors)) %>%
  bind_cols(
    as.data.frame(pred_lasso) %>% rename_with(~ paste0("lasso_", .)),
    as.data.frame(pred_rf) %>% rename_with(~ paste0("rf_", .)),
    as.data.frame(pred_nn) %>% rename_with(~ paste0("nn_", .))
  ) %>%
  slice((train_window + 1):n) # keep only out-of-sample rows

cat("Out-of-sample predictions stored:", nrow(results), "rows\n")
```

```
## Out-of-sample predictions stored: 347 rows
```

5: Evaluation Matrices

```

# 5.1 Out-of-Sample  $R^2$  function

oos_r2 <- function(actual, predicted) {
  benchmark <- mean(actual, na.rm = TRUE)
  mse_model <- mean((actual - predicted)^2, na.rm = TRUE)
  mse_benchmark <- mean((actual - benchmark)^2, na.rm = TRUE)
  1 - mse_model / mse_benchmark
}

# 5.2 Compute  $R^2_{oos}$  for each model  $\times$  factor  $\times$  regime

models <- c("lasso", "rf", "nn")
regimes <- c("full", "low_vol", "high_vol")

r2_results <- expand_grid(model = models,
                          factor = factors,
                          regime = regimes,
                          stringsAsFactors = FALSE) %>%
  mutate(r2_oos = NA_real_)

for (i in seq_len(nrow(r2_results))) {
  mod <- r2_results$model[i]
  fac <- r2_results$factor[i]
  reg <- r2_results$regime[i]

  if (reg == "full") {
    sub <- results
  } else {
    sub <- results %>% filter(regime == reg)
  }

  actual <- sub[[paste0("actual_", fac)]]
  predicted <- sub[[paste0(mod, "_", fac)]]

  r2_results$r2_oos[i] <- oos_r2(actual, predicted)
}

cat("\nOut-of-Sample  $R^2$  by Model and Factor\n")

```

```

##
## Out-of-Sample  $R^2$  by Model and Factor

```

```

r2_wide <- r2_results %>%
  filter(regime == "full") %>%
  select(model, factor, r2_oos) %>%
  pivot_wider(names_from = factor, values_from = r2_oos) %>%
  mutate(across(where(is.numeric), ~ round(., 4)))
print(r2_wide)

```

```

## # A tibble: 3 x 7
##   model   MKT   SMB   HML   RMW   CMA   MOM
##   <chr> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>

```

```
## 1 lasso -0.177 -0.254 -0.210 -0.0823 -0.181 -0.0439
## 2 rf     -0.175 -0.112 -0.129 -0.0601 -0.0912 -0.113
## 3 nn     -0.536 -0.453 -0.297 -0.324 -0.250 -0.547
```

```
cat("\nR2_oos by Regime\n")
```

```
##
## R2_oos by Regime
```

```
r2_regime <- r2_results %>%
  filter(regime != "full") %>%
  group_by(model, regime) %>%
  summarise(avg_r2 = round(mean(r2_oos), 4), .groups = "drop")
print(r2_regime)
```

```
## # A tibble: 6 x 3
##   model regime   avg_r2
##   <chr> <chr>     <dbl>
## 1 lasso high_vol -0.188
## 2 lasso low_vol  -0.145
## 3 nn    high_vol -0.423
## 4 nn    low_vol  -0.477
## 5 rf    high_vol -0.108
## 6 rf    low_vol  -0.237
```

```
# 5.3 Sharpe Ratio of Long-Short Factor Rotation Strategy
# Each month: go long the factor with highest predicted return,
#             short the factor with lowest predicted return
```

```
sharpe <- function(returns) {
  mean(returns, na.rm = TRUE) / sd(returns, na.rm = TRUE) * sqrt(12)
}

strategy_returns <- data.frame(date = results$date, regime = results$regime)

for (mod in models) {
  pred_cols <- paste0(mod, "_", factors)
  strat_returns <- numeric(nrow(results))
  for (j in seq_len(nrow(results))) {
    preds <- as.numeric(results[j, pred_cols])
    long_ret <- results[j, paste0("actual_", factors[which.max(preds)])]
    short_ret <- results[j, paste0("actual_", factors[which.min(preds)])]
    strat_returns[j] <- as.numeric(long_ret) - as.numeric(short_ret)
  }

  strategy_returns[[mod]] <- strat_returns

  cat("\nSharpe Ratio -", toupper(mod), "strategy:", round(sharpe(strat_returns), 3))
  cat("(low_vol:", round(sharpe(strat_returns[results$regime == "low_vol"]), 3), ")")
  cat("(high_vol:", round(sharpe(strat_returns[results$regime == "high_vol"]), 3), ")\n")
}
```

```
##
## Sharpe Ratio - LASSO strategy: 0.219(low_vol: 0.705 )(high_vol: 0.047 )
##
## Sharpe Ratio - RF strategy: 0.198(low_vol: 0.777 )(high_vol: -0.031 )
##
## Sharpe Ratio - NN strategy: 0.221(low_vol: 0.699 )(high_vol: 0.045 )
```

5.4 Hit Ratio

Proportion of months where predicted direction matches actual direction

```
hit_ratio <- function(actual, predicted) {
  correct <- sign(actual) == sign(predicted)
  mean(correct, na.rm = TRUE)
}

hit_results <- expand_grid(model = models,
                          factor = factors,
                          regime = regimes,
                          stringsAsFactors = FALSE) %>%
  mutate(hit = NA_real_)

for (i in seq_len(nrow(hit_results))) {
  mod <- hit_results$model[i]
  fac <- hit_results$factor[i]
  reg <- hit_results$regime[i]

  if (reg == "full") {
    sub <- results
  } else {
    sub <- results %>% filter(regime == reg)
  }

  actual <- sub[[paste0("actual_", fac)]]
  predicted <- sub[[paste0(mod, "_", fac)]]

  hit_results$hit[i] <- hit_ratio(actual, predicted)
}

cat("\nHit Ratio by Model and Factor (Full Sample)\n")
```

```
##
## Hit Ratio by Model and Factor (Full Sample)
```

```
hit_wide <- hit_results %>%
  filter(regime == "full") %>%
  select(model, factor, hit) %>%
  pivot_wider(names_from = factor, values_from = hit) %>%
  mutate(across(where(is.numeric), ~ round(., 4)))
print(hit_wide)
```

```
## # A tibble: 3 x 7
##   model  MKT   SMB   HML   RMW   CMA   MOM
##   <chr> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
```

```
## 1 lasso 0.594 0.513 0.516 0.516 0.487 0.579
## 2 rf    0.559 0.478 0.504 0.499 0.542 0.499
## 3 nn    0.559 0.487 0.545 0.510 0.470 0.553
```

```
cat("\nAverage Hit Ratio by Model and Regime\n")
```

```
##
## Average Hit Ratio by Model and Regime
```

```
hit_regime <- hit_results %>%
  filter(regime != "full") %>%
  group_by(model, regime) %>%
  summarise(avg_hit = round(mean(hit), 4), .groups = "drop")
print(hit_regime)
```

```
## # A tibble: 6 x 3
##   model regime   avg_hit
##   <chr> <chr>     <dbl>
## 1 lasso high_vol  0.512
## 2 lasso low_vol  0.559
## 3 nn    high_vol  0.503
## 4 nn    low_vol  0.542
## 5 rf    high_vol  0.506
## 6 rf    low_vol  0.522
```

6: Plots

```
# 6.1 Cumulative Returns of Long-Short Strategy by Model
# Build strategy return series for each model

# Cumulative returns
cum_returns <- strategy_returns %>%
  mutate(across(all_of(models), ~ cumsum(.))) %>%
  pivot_longer(cols = all_of(models), names_to = "model", values_to = "cum_return") %>%
  mutate(model = toupper(model))

regime_bands <- results %>%
  mutate(xmin = date,
         xmax = lead(date, default = last(date)),
         fill_color = ifelse(regime == "high_vol", "High Volatility", "Low Volatility"))

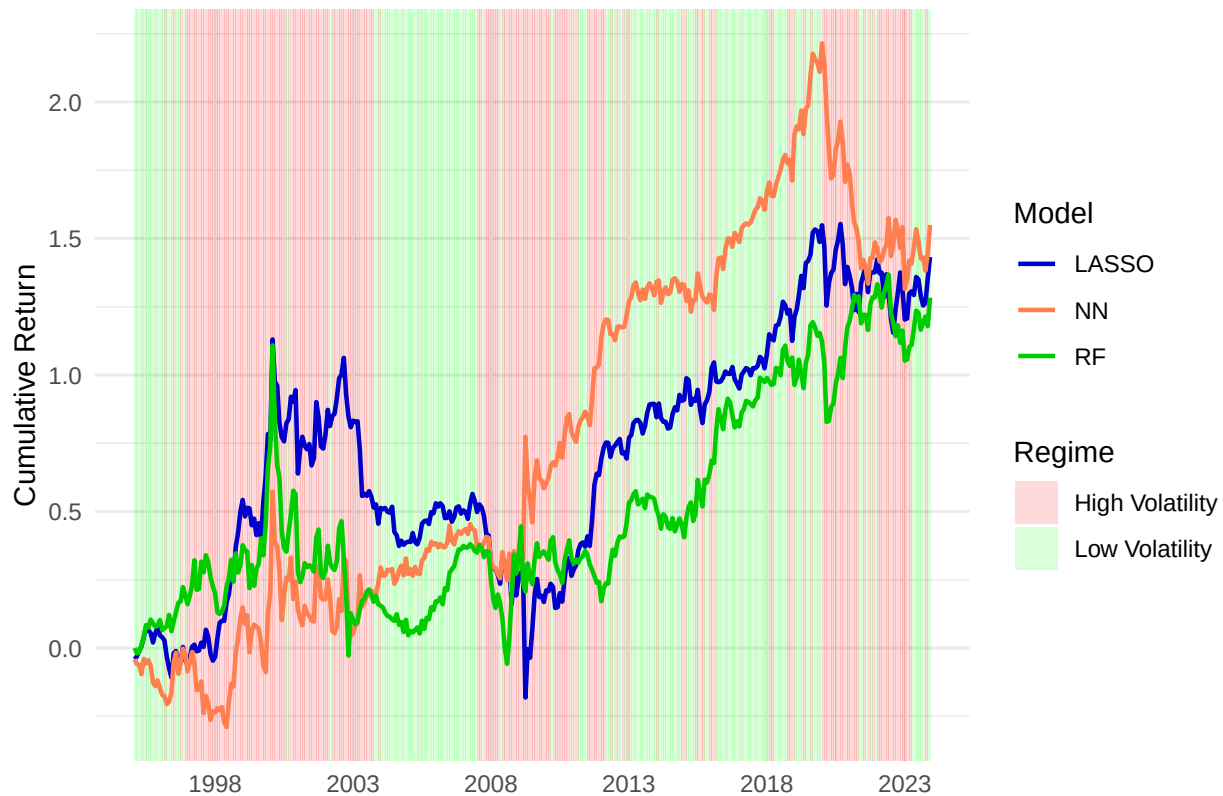
p1 <- ggplot(cum_returns, aes(x = date, y = cum_return, color = model)) +
  geom_rect(data = regime_bands,
           aes(xmin = xmin, xmax = xmax, ymin = -Inf, ymax = Inf, fill = fill_color),
           inherit.aes = FALSE, alpha = 0.15) +
  geom_line(linewidth = 0.8) +
  labs(title = "Cumulative Returns of Long-Short Factor Rotation Strategy",
       x = NULL, y = "Cumulative Return", color = "Model", fill = "Regime") +
  theme_minimal() +
  scale_color_manual(values = c("LASSO" = "blue3",
```

```

      "RF" = "green3",
      "NN" = "coral")) +
scale_fill_manual(values = c("Low Volatility" = "green",
                             "High Volatility" = "red")) +
scale_x_date(date_breaks = "5 years", date_labels = "%Y",
             limits = c(min(results$date), max(results$date)))
print(p1)

```

Cumulative Returns of Long–Short Factor Rotation Strategy



```

# 6.2 Rolling 12-Month Sharpe Ratio by Model over Time

roll_window <- 12

rolling_sharpe <- data.frame(date = results$date)

for (mod in models) {
  ret <- strategy_returns[[mod]]
  rs <- numeric(length(ret))
  for (j in seq_along(ret)) {
    if (j < roll_window) {
      rs[j] <- NA
    } else {
      window_ret <- ret[(j - roll_window + 1):j]
      rs[j] <- mean(window_ret) / sd(window_ret) * sqrt(12)
    }
  }
}

```



```

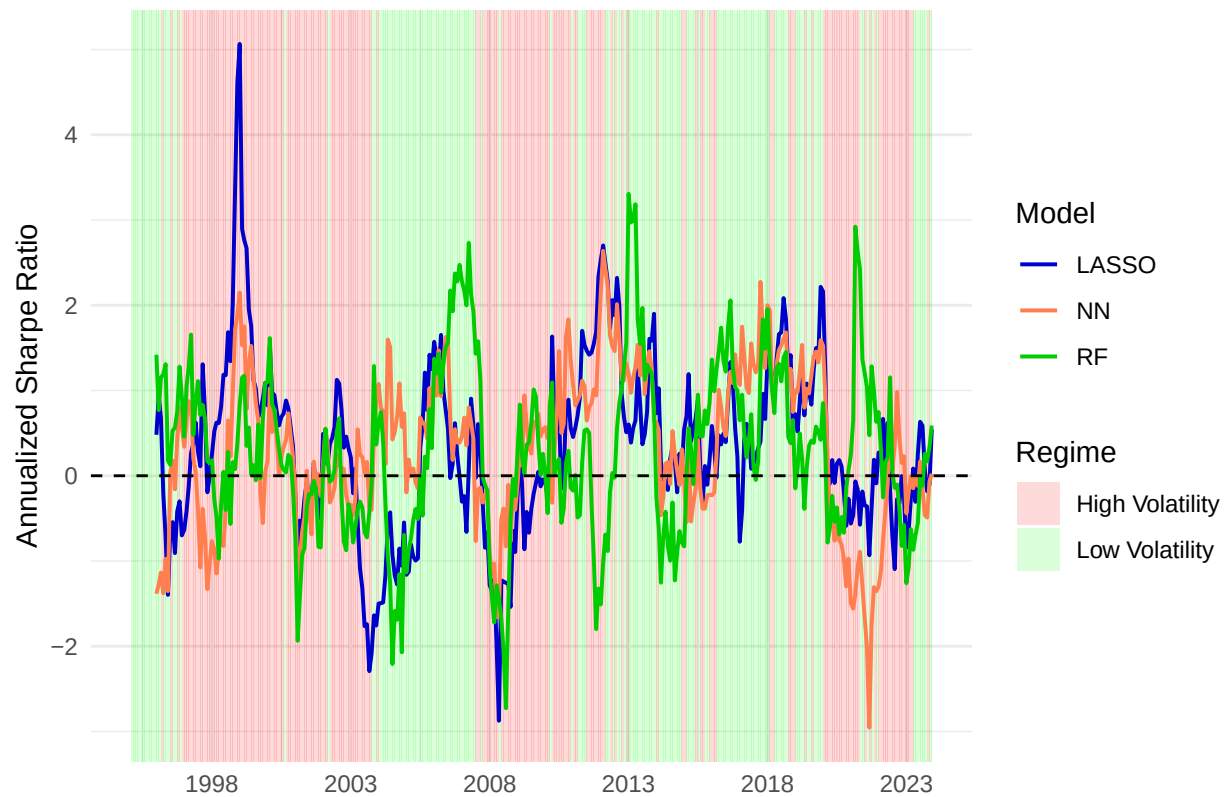
    rolling_sharpe[[toupper(mod)]] <- rs
  }

rolling_sharpe_long <- rolling_sharpe %>%
  pivot_longer(cols = toupper(models), names_to = "model", values_to = "sharpe") %>%
  left_join(results %>% select(date, regime), by = "date") %>%
  filter(!is.na(sharpe))

p2 <- ggplot(rolling_sharpe_long, aes(x = date, y = sharpe, color = model)) +
  geom_rect(data = regime_bands,
            aes(xmin = xmin, xmax = xmax, ymin = -Inf, ymax = Inf, fill = fill_color),
            inherit.aes = FALSE, alpha = 0.15) +
  geom_line(linewidth = 0.7) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "black") +
  labs(title = "Rolling 12-Month Sharpe Ratio by Model",
       x = NULL, y = "Annualized Sharpe Ratio",
       color = "Model", fill = "Regime") +
  theme_minimal() +
  scale_color_manual(values = c("LASSO" = "blue3",
                                "RF" = "green3",
                                "NN" = "coral")) +
  scale_fill_manual(values = c("Low Volatility" = "green",
                                "High Volatility" = "red")) +
  scale_x_date(date_breaks = "5 years", date_labels = "%Y",
               limits = c(min(results$date), max(results$date)))
print(p2)

```

Rolling 12-Month Sharpe Ratio by Model



6.3 Rolling 12-Month Hit Ratio by Model over Time

```
rolling_hit <- data.frame(date = results$date)

for (mod in models) {
  rh <- numeric(nrow(results))
  for (j in seq_len(nrow(results))) {
    if (j < roll_window) {
      rh[j] <- NA
    } else {
      window_idx <- (j - roll_window + 1):j
      # Compute hit ratio per factor first, then average across factors
      hit_per_factor <- sapply(factors, function(f) {
        actual <- results[[paste0("actual_", f)]] [window_idx]
        predicted <- results[[paste0(mod, "_", f)]] [window_idx]
        mean(sign(actual) == sign(predicted), na.rm = TRUE)
      })
      rh[j] <- mean(hit_per_factor, na.rm = TRUE)
    }
  }
  rolling_hit[[toupper(mod)]] <- rh
}

rolling_hit_long <- rolling_hit %>%
  pivot_longer(cols = toupper(models), names_to = "model", values_to = "hit") %>%
  left_join(results %>% select(date, regime), by = "date") %>%
```

```

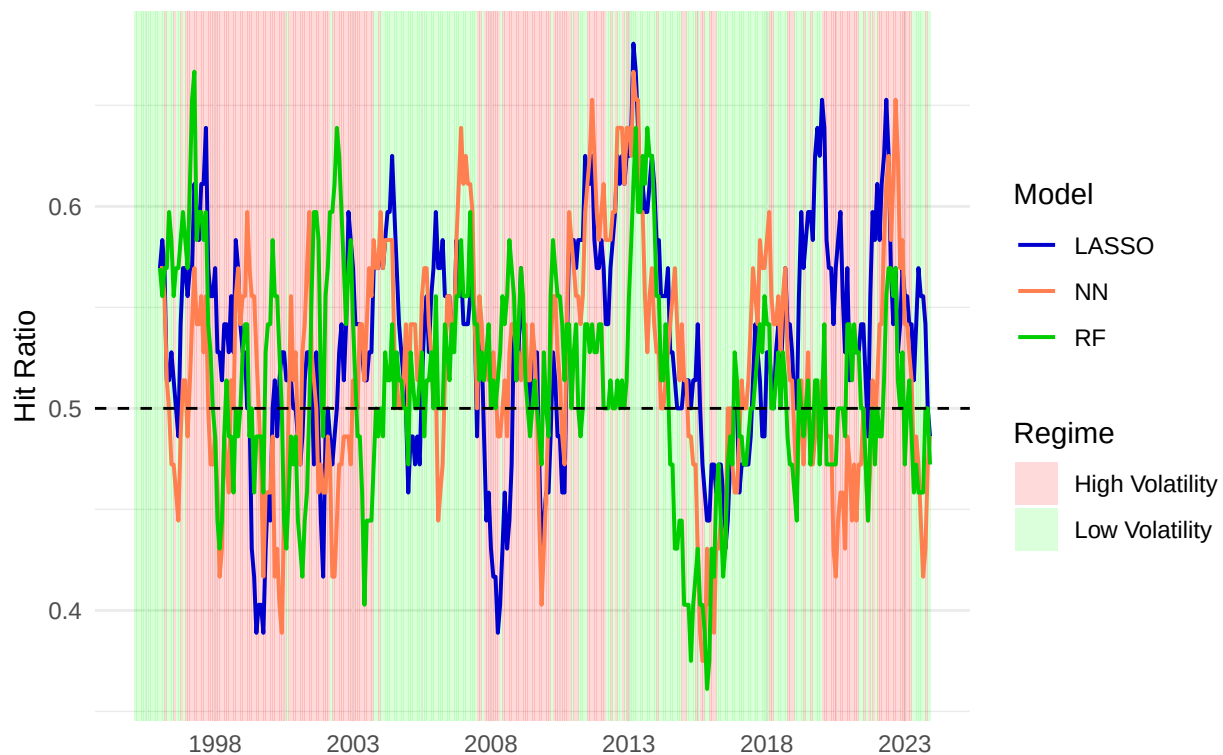
filter(!is.na(hit))

p3 <- ggplot(rolling_hit_long, aes(x = date, y = hit, color = model)) +
  geom_rect(data = regime_bands,
            aes(xmin = xmin, xmax = xmax, ymin = -Inf, ymax = Inf, fill = fill_color),
            inherit.aes = FALSE, alpha = 0.15) +
  geom_line(linewidth = 0.7) +
  geom_hline(yintercept = 0.5, linetype = "dashed", color = "black") +
  labs(title = "Rolling 12-Month Hit Ratio by Model",
        subtitle = "Dashed = 0.5 random benchmark",
        x = NULL, y = "Hit Ratio", color = "Model", fill = "Regime") +
  theme_minimal() +
  scale_color_manual(values = c("LASSO" = "blue3",
                                "RF" = "green3",
                                "NN" = "coral")) +
  scale_fill_manual(values = c("Low Volatility" = "green",
                                "High Volatility" = "red")) +
  scale_x_date(date_breaks = "5 years", date_labels = "%Y",
               limits = c(min(results$date), max(results$date)))
print(p3)

```

Rolling 12-Month Hit Ratio by Model

Dashed = 0.5 random benchmark



6.4 Rolling 12-Month Out-of-Sample R^2 by Model over Time

```
rolling_r2 <- data.frame(date = results$date)
```

```

for (mod in models) {
  rr <- numeric(nrow(results))
  for (j in seq_len(nrow(results))) {
    if (j < roll_window) {
      rr[j] <- NA
    } else {
      window_idx <- (j - roll_window + 1):j
      r2_vals <- sapply(factors, function(f) {
        actual <- results[[paste0("actual_", f)]] [window_idx]
        predicted <- results[[paste0(mod, "_", f)]] [window_idx]
        benchmark <- mean(actual, na.rm = TRUE)
        1 - mean((actual - predicted)^2) / mean((actual - benchmark)^2)
      })
      rr[j] <- mean(r2_vals, na.rm = TRUE)
    }
  }
  rolling_r2[[toupper(mod)]] <- rr
}

rolling_r2_long <- rolling_r2 %>%
  pivot_longer(cols = toupper(models), names_to = "model", values_to = "r2") %>%
  filter(!is.na(r2))

p4 <- ggplot(rolling_r2_long, aes(x = date, y = r2, color = model)) +
  geom_rect(data = regime_bands,
            aes(xmin = xmin, xmax = xmax, ymin = -Inf, ymax = Inf, fill = fill_color),
            inherit.aes = FALSE, alpha = 0.15) +
  geom_line(linewidth = 0.7) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "black") +
  labs(title = "Rolling 12-Month Out-of-Sample R2 by Model",
       subtitle = "Dashed = historical mean benchmark",
       x = NULL, y = "Average R2_oos (across factors)", color = "Model", fill = "Regime") +
  theme_minimal() +
  scale_color_manual(values = c("LASSO" = "blue3",
                                "RF" = "green3",
                                "NN" = "coral")) +
  scale_fill_manual(values = c("Low Volatility" = "green",
                                "High Volatility" = "red")) +
  scale_x_date(date_breaks = "5 years", date_labels = "%Y",
               limits = c(min(results$date), max(results$date))) +
  coord_cartesian(ylim = c(-5, 0.1))
print(p4)

```

Rolling 12-Month Out-of-Sample R^2 by Model

Dashed = historical mean benchmark

